Part-1) Prime Number Counter FSM

States

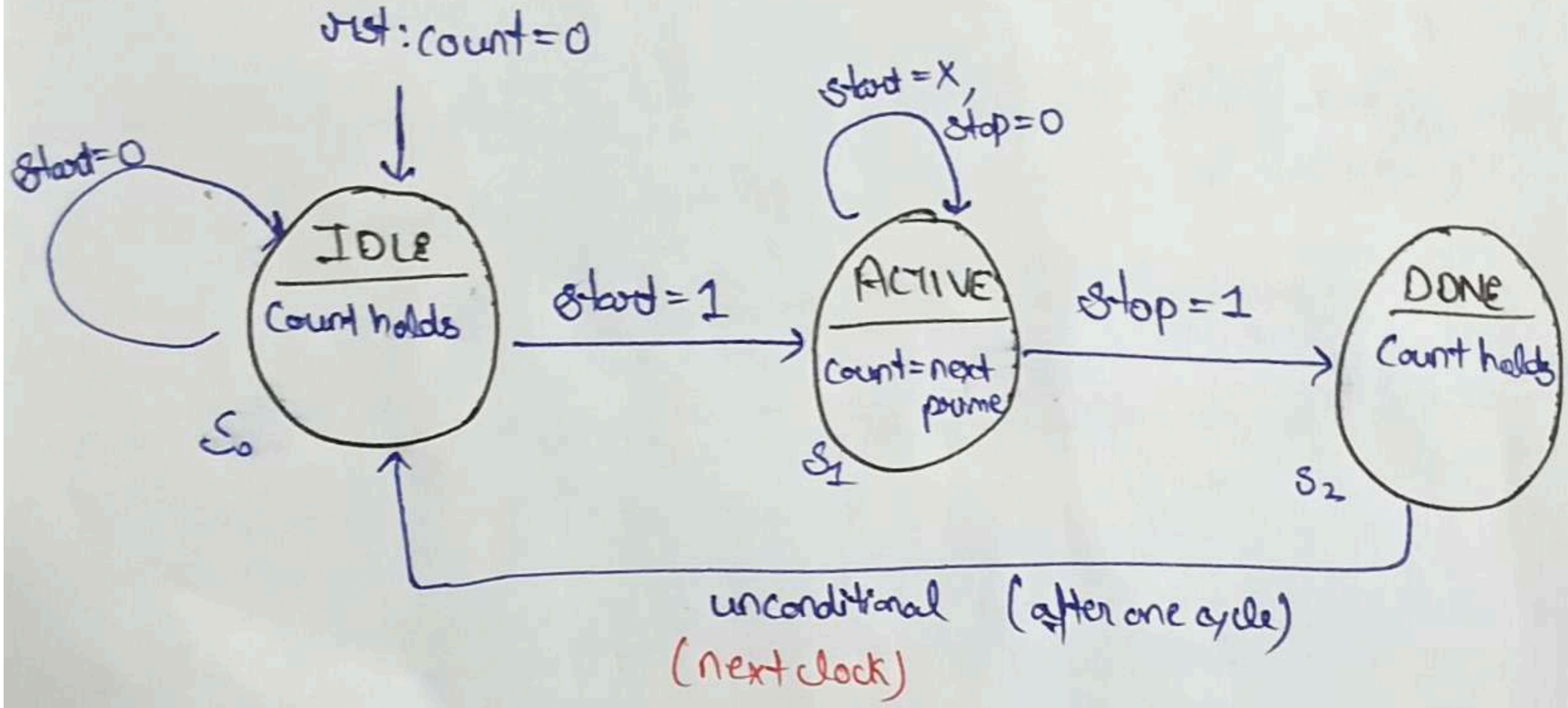
S_0: IDLE (count hold its value)

S_1: ACTIVE (count: 2 → 3 → 5 → 7 → 11 → 13 → 2 ...)

S_2: DONE (it will exist for exactly one clock cycle and then go to IDLE)

"unconditional"

State Diagram



IDLE:- Right after reset (in any state), it holds 0, and after a stop it holds the last prime.

ACTIVE: Whenever STOP=0, the count moves (2, 3, 5, 7, 11, 13, 2, ...)

If STOP=1 at any count value, it will freeze and hold that value.

It move to DONE.

DONE → IDLE: No input needed, (count holds its value)

Verilog code

```
module part_1 ( input wire clk,
                input wire rst,
                input wire start,
                input wire stop,
                output reg [1:0] state,
                output reg [3:0] count);
```

// module declaration

```
parameter IDLE = 2'b00, ACTIVE = 2'b01, DONE = 2'b10;
parameter S0 = 2'b00, S1 = 2'b01, S2 = 2'b10;
reg [1:0] next_state;

always @ (posedge clk) begin
    if (rst)
        state <= S0;
    else
        state <= next_state;
end
```

state register code

```
always @ (*) begin
    case (state)
    S0: begin
        if (start)
            next_state = S1;
        else
            nextstate = S0;
    end
    S1 : begin
        if (stop)
            next_state = S2;
        else
            next_state = S1;
    end
    S2 : begin
            next_state = S0;
```

It is next state logic using blocking assignments.

```
default : next_state = S0;
endcase.
end
```

```
always @(posedge clk)  begin
    if (rst)
    count <= 4'd0;
     else begin
  S0: begin
      if (start)
          count <= 4'd2;
    end
  S1: begin
    if (!stop)  begin
        case (count)
     4'd2 : count <= 4'd3;
     4'd3 : count <= 4'd5;
     4'd5 : count <= 4'd7;
     4'd7 : count <= 4'd11;
     4'd11 : count <= 4'd13;
     4'd13 : count <= 4'd2;
      default : count <= 4'd2;
       endcase
    end
    S2 : begin
           end
        end
      default : count <= 4'd0;
       endcase
    end
  end
endmodule
```

Counter block

No else statement
∴ it will latch
||
hold count

Here also no else, statement
This means if Stop = 1, it will hold the count.

Bonus (without using 'lookup' method)

What is loopup method ?

⇒ in ACTIVE Counter block,

Count <= next_prime (Count);

using for loop,

a function is_prime can be used.

Code

```
function is_prime;
    input integer n;
  integer d;
   begin
        is_prime = (n>=2);
    for (d=2; d<n ; d=d+1)
        if (n % d == 0)
            is_prime = 0;
   end
end function
```

Part-2 : Shared Resource Arbiter

Version 1: "Fixed Priority", When both request, A wins

Version 2: "Fair Arbiter", Round-robin: remember who was granted last, and on a tie the other module wins.

⇒ For version 1, we don't need memory to store previous output, it is pure combinational logic.

For version 2, we need to store ⇒ who was granted last.

Interface

clk : clock
rst : Synchronous, active high reset
req-A, req-B : Request from modules A and B
data-A, data-B : 8-bit data from each module
grant-A, grant-B: Indicates which module is selected.
out-data : selected module's 8-bit data.
out-valid : 1 when a module is granted, 0 otherwise.

For fair arbiter

last_B: Stores the history whether B was granted in the previous cycle.

next_B: temporary variable that calculates what the history should become after the current grant.

Readme answers

①

Cycle	Fixed Pr	Fair
1	A	A wins initially
2	A	B wins
3	A	A wins
4	A	B wins

② If both req remains high, A gets every grant and B may wait indefinitely. In a 20-cycle test with both request held high, the fixed priority arbiter gives B → 0 grants; while the fair arbiter gives B 10 grants.

③ The Round Robin Arbiter remembers the last granted module and gives priority to the other module when both request. If both keep requesting, each gets a grant every two cycles. So the maximum wait b/w grant is 2 cycles.

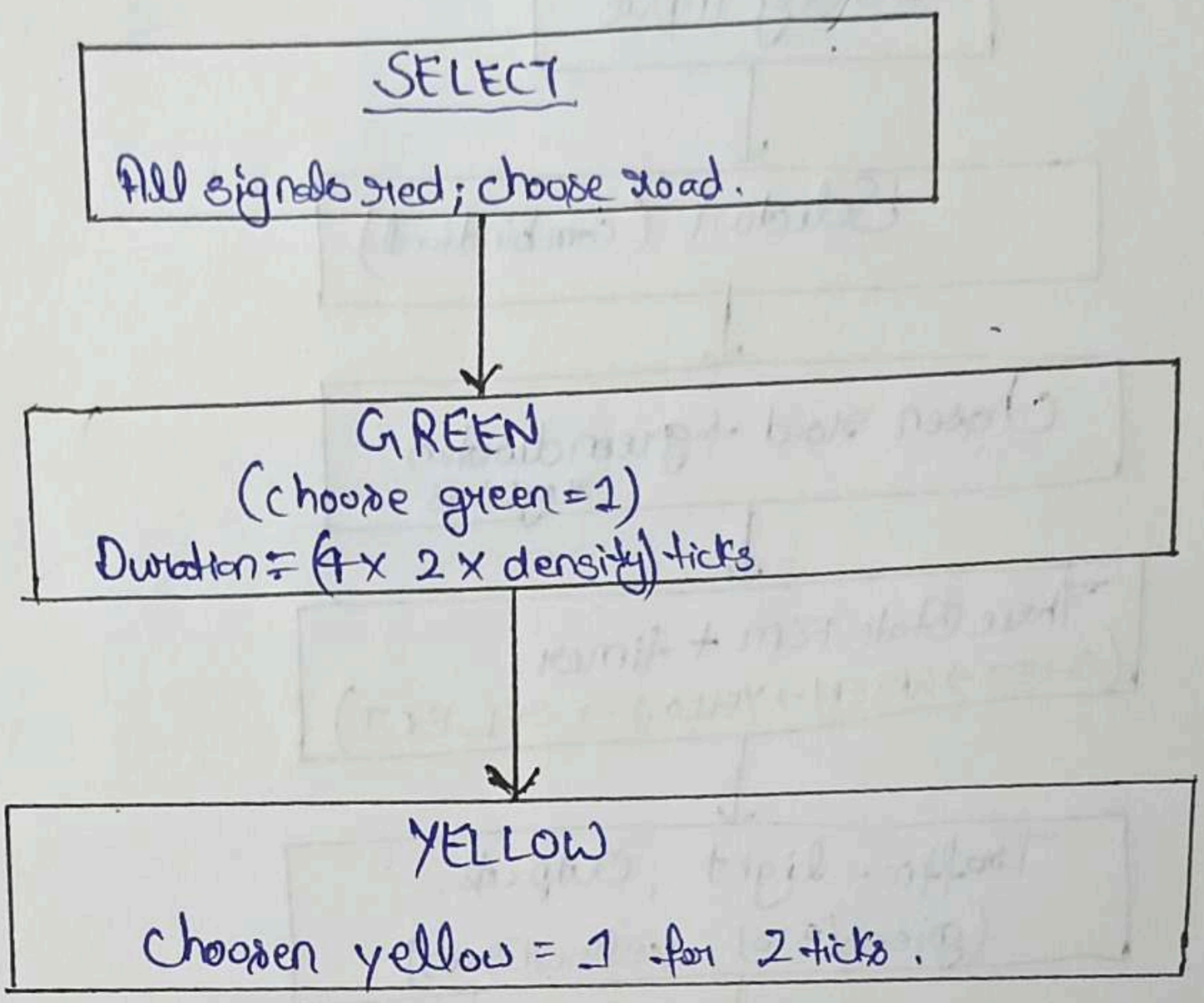
Part-3: Smart Crossroad Controllers

Rule①| Busiest wins

Rule②: Ties alternate (On a tie, choose the next tied dir after the one served last) order ⇒ N, E, S, W.

Rule③: No Starvation

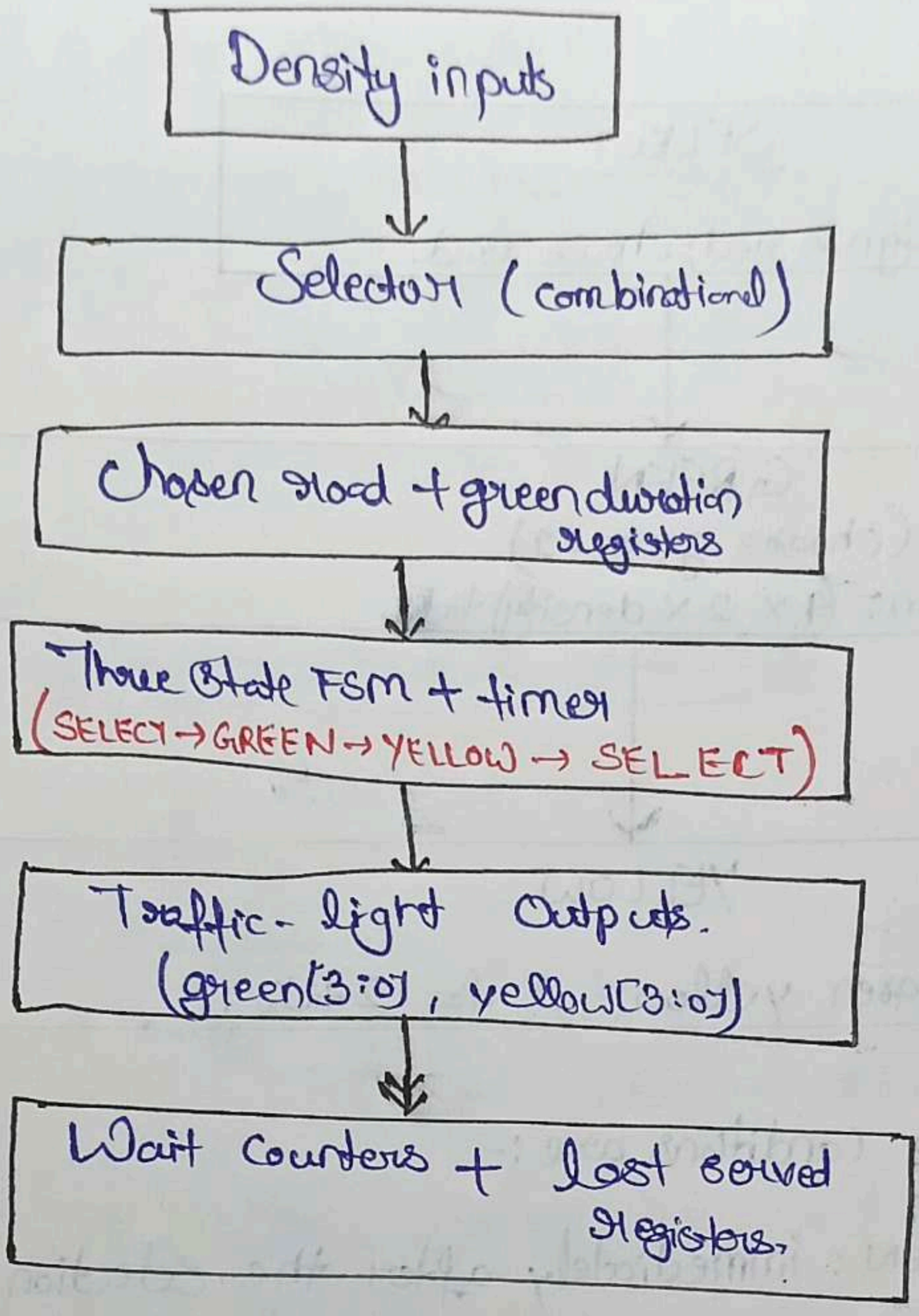
State Diagram



The transition conditions are :-

1) SELECT → GREEN : immediately after the selection tick
2) GREEN → YELLOW : when the green timer reaches the stored duration
3) YELLOW → SELECT : when two ticks have occured.

Block Diagram



Assumptions and register encoding

Reset ⇒ sync and active high. (rst)

Last served after reset ⇒ LS=2 W (2'd3) so N wins the first tie.

Direction encoding → N=0, E=1, S=2, W=3

Wait counter : 3-bit Saturating counter. (max ⇒ 7)

Wait counter update ⇒ at each SELECT clock edge

Forced-wait tie ⇒ Highest wait count wins.

Density sample ⇒ only when in SELECT.

Outputs in SELECT ⇒ all red.